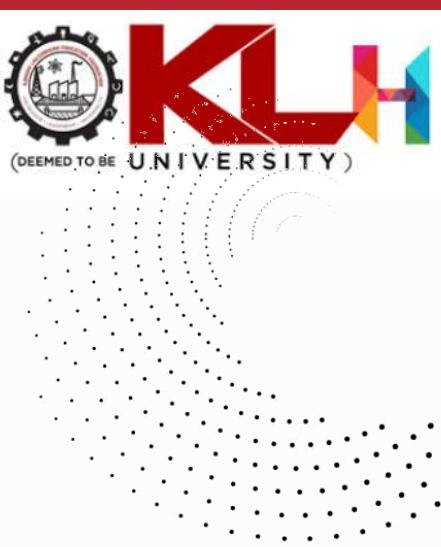




APPLIED MACHINE LEARNING FOR TEXT ANALYSIS (24ALT3101)

AI Copilot Code Review using Large Language Models

Team Members

Name: BANDI SRIHITHA Rollno: 2420030099

Name: EMANI MAHATHI Rollno: 2420030093

Name: PRATHIPATI PRANATHI Rollno: 2420030043

Under the Esteemed Guidance of

Dr . KATANGURI SWANTHANA (CSE)

Contents

- Problem Statement
- Objectives
- Literature Survey
- Research Gap
- Innovation & Creativity
- Novelty
- Feasibility
- Project Plan
- Proposed Architecture/Methodology
- Datasets & Technology Stack
- References

PROBLEM STATEMENT

Problem Statement

Software code review is essential for ensuring code quality, security, and maintainability. Traditional static analysis tools detect rule-based issues but lack contextual understanding, while Large Language Models (LLMs) generate intelligent suggestions but may produce inaccurate or unsupported recommendations. Existing approaches also lack repository awareness and issue prioritization. Therefore, an intelligent hybrid framework is required to generate accurate, explainable, and context-aware automated code reviews.

Motivation

- Manual code reviews are time-consuming.
- Static analyzers lack contextual understanding.
- LLMs may generate hallucinated suggestions.
- Existing systems ignore repository context.
- Developers need explainable AI-assisted reviews.

OBJECTIVES

Objectives:

- **To develop** an AI-powered automated code review system using **CodeLlama** to generate intelligent and context-aware review suggestions.
- **To enable** automated code review for **Python, Java, and JavaScript** using language-specific static analysis tools.
- **To integrate CodeBERT, FAISS, and Retrieval-Augmented Generation (RAG)** to retrieve relevant code context and improve review accuracy.
- **To generate** explainable review comments with **issue descriptions, severity levels, confidence scores, and recommended code fixes**.
- **To evaluate** the proposed framework using **CodeSearchNet, CodeXGLUE, and GitHub repositories** to measure review quality and overall system performance.

LITERATURE SURVEY

Research Paper	Method	Advantages	Limitations
AI-powered Code Review with LLMs (2024)	LLM-based Review	Intelligent suggestions	Limited repository context
Security Code Review using LLMs (2024)	GPT + Security Analysis	Better vulnerability detection	High false positives
Evaluating Source Code Quality with LLMs (2024)	LLM Assessment	Improved maintainability	No static analysis
Hybrid LLM + Static Analysis (2025)	Static Analysis + RAG	Better review quality	Limited language support
Automated Code Review with Symbolic Reasoning (2025)	LLM + Reasoning	Better explanations	Higher computation

Common Research Challenges

- Limited contextual code understanding.
- Generic review recommendations.
- Weak integration of review techniques.
- Lack of issue prioritization, Limited multi-language support.

RESEARCH GAP

Research Gap	Proposed Solution
Limited understanding of repository context	Retrieve repository context using CodeBERT and FAISS
Generic and inconsistent review suggestions	Generate context-aware reviews using CodeLlama
Static analysis lacks semantic reasoning	Combine static analysis with LLM reasoning
No severity-based issue prioritization	Rank issues as Critical, High, Medium, and Low
Limited explainability of review comments	Provide reasons, fixes, and confidence scores

Innovation

- Repository-aware code review using FAISS-based RAG.
- Hybrid integration of CodeLlama and static analysis.
- Intelligent retrieval of similar code before review.
- Context-aware code review with explainable suggestions.
- Unified review report with prioritized issue severity.

Creativity

- Simulates expert developer review workflow.
- Combines rule-based and AI-driven analysis.
- Supports Python, Java, and JavaScript.
- Generates developer-friendly review explanations.
- Modular architecture for future expansion.

NOVALITY

Existing System	Proposed System
Generic LLM review	Context-aware AI review
Static analysis only	Hybrid AI + static analysis
No repository context	CodeBERT + FAISS retrieval
Generic suggestions	Explainable recommendations
Equal issue priority	Severity-ranked review

Key Contribution

- Hybrid AI-powered code review framework.
- CodeBERT-based semantic code retrieval.
- Repository-aware contextual review generation.
- Explainable and severity-ranked suggestions.
- Multi-language support for three languages.

FEASIBILITY ANALYSIS

Technical Feasibility

- Open-source AI models and libraries available.
- Supports Python, Java, and JavaScript.
- Efficient retrieval using CodeBERT and FAISS.
- Easy integration with static analysis tools.
- Achievable within the proposed timeline.

Economic Feasibility

- Uses open-source technologies.
- No expensive model training required.
- Low development and deployment cost.

Operational Feasibility

- User-friendly Streamlit interface.
- Easy integration into developer workflow.
- Modular and scalable architecture.

Time Feasibility

- Prototype completed within **4 weeks**.
- Based on existing AI and RAG technologies.

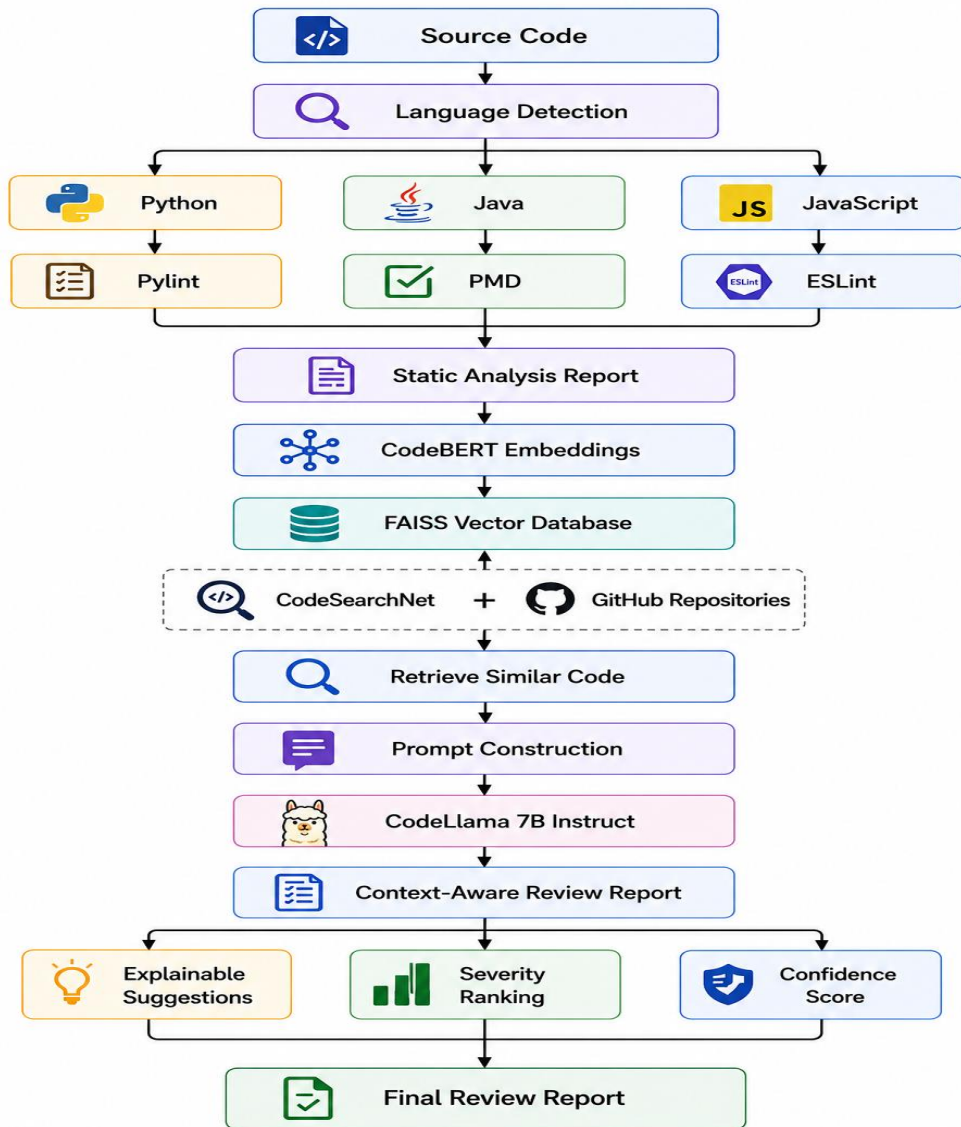
PROJECT PLAN

Week	Activity
Week 1	Literature Survey & Dataset Collection
Week 1	Code Chunking & FAISS Setup
Week 2	Static Analyzer Integration
Week 2	CodeBERT & CodeLlama Integration
Week 3	RAG Implementation & Prompt Engineering
Week 3	Explainable Review Generation
Week 4	Testing & Performance Evaluation
Week 4	Documentation & Final Presentation

Expected Outcomes

- Automate code review for Python, Java, and JavaScript.
- Improve code quality through context-aware analysis.
- Generate accurate and explainable review suggestions.
- Prioritize issues based on severity levels.
- Reduce manual code review effort and time.
- Enhance developer productivity and code maintainability.
- Provide a scalable framework for future language support.

PROPOSED ARCHITECTURE/METHODOLOGY



Workflow

1. Upload source code.
2. Detect programming language.
3. Perform static code analysis.
4. Generate CodeBERT embeddings.
5. Retrieve similar code using FAISS.
6. Build contextual prompt.
7. Generate review using CodeLlama.
8. Produce explainable review report.

DATA SET

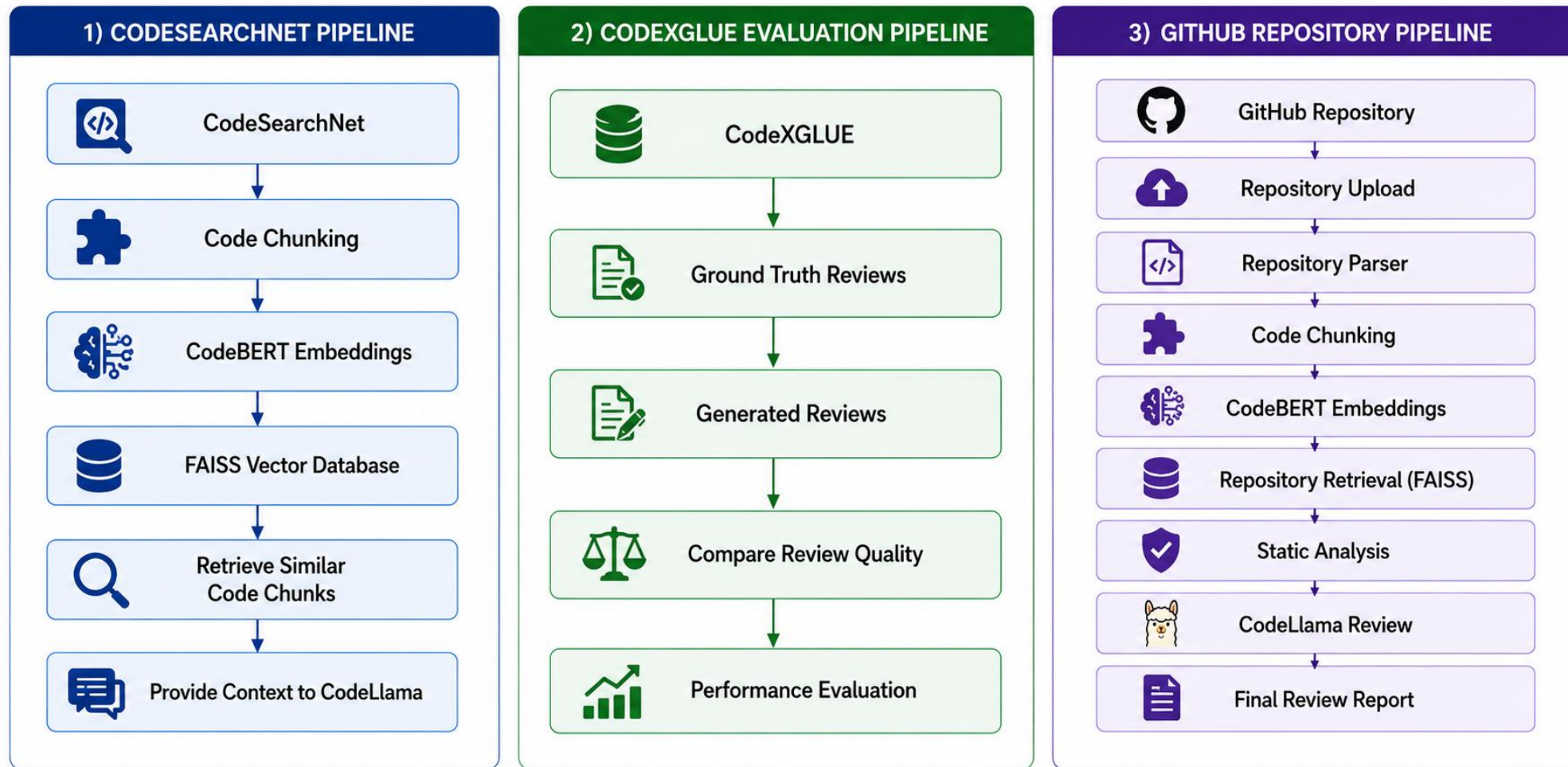
Dataset	Purpose	Used In	links
CodeSearchNet	Similar code repository	CodeBERT embeddings & FAISS retrieval	https://github.com/github/CodeSearchNet
CodeXGLUE	Code review benchmark	Performance evaluation	https://github.com/microsoft/CodeXGLUE
GitHub Public Repositories	Repository context	RAG retrieval & testing	https://github.com/?utm_source=chatgpt.com

Technology Stack:

- **Programming Language:** Python
- **User Interface:** Streamlit
- **Code Large Language Model:** CodeLlama
- **Embedding Model:** CodeBERT
- **Vector Database:** FAISS
- **Static Analysis Tools:** Pylint, PMD, ESLint
- **Retrieval Technique:** Retrieval-Augmented Generation (RAG)
- **Datasets:** CodeSearchNet, CodeXGLUE, GitHub Public Repositories
- **Evaluation Metrics:** Precision, Recall, CodeBLEU

Flow Chart

How Each Dataset Fits Into Our Workflow?



REFERENCES

- B. Rozière *et al.*, "Code Llama: Open Foundation Models for Code," Meta AI, 2023.
- Z. Feng *et al.*, "CodeBERT: A Pre-Trained Model for Programming and Natural Languages," *Findings of EMNLP*, 2020.
- S. Lu *et al.*, "CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation," *NeurIPS Datasets and Benchmarks*, 2021.
- H. Husain *et al.*, "CodeSearchNet Challenge: Evaluating the State of Semantic Code Search," *Empirical Software Engineering*, 2021.
- J. Johnson, M. Douze, and H. Jégou, "Billion-scale Similarity Search with GPUs (FAISS)," *IEEE Transactions on Big Data*, 2019.
- P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- "AI-powered Code Review with Large Language Models: Early Results," 2024.
- "An Insight into Security Code Review with Large Language Models," 2024.
- "Evaluating Source Code Quality with Large Language Models," 2024.
- "Combining Large Language Models with Static Analysis for Automated Code Review," 2025.

Thankyou